STRIDE Threat Model: SAFU Protection Pool (Stellar, multichain)

Prepared for: the security auditor and the Stellar Development Foundation, Soroban Audit Bank intake **System:** SAFU protection pool: five Soroban smart contracts (Rust → WASM) and their off-chain signers **Repository:** `github.com/mrkanchwala/safu_multichain` (public), branch `main` **Tree modelled:** `main`, contract code at commit `6e61cf4` (pinned with a tag once the pool is on testnet) **Contract source:** `contracts/protection-pool`, `covered-registry`, `cctp-adapter`, `cctp-common`, `safu-account`: 5,612 functional SLOC **Date:** 2026-10-20 **Framework:** STRIDE, following SDF's published threat-modelling guidance **Status:** self-assessment prepared as an audit input. This is not an audit.

What are we working on?

Project description

SAFU is a **loss-socialization protocol**: not insurance, and it collects no premium.

A staker deposits USDC into a shared protection pool, nominates a beneficiary address, and names up to three wallets to cover. If a covered wallet is later drained (by phishing, an approval exploit, or key compromise), SAFU's off-chain fraud scanner scores the incident, and the pool pays out a deterministic entitlement of up to a tier multiple of the stake. The staker's principal is **forfeited** on a paid claim. Backers add capital that counts toward the pool's capacity, earns yield and is never forfeited. There is no premium, no underwriter, no human vote, and no discretionary claims committee.

The contracts modelled here are the on-chain half. They hold staker and backer principal, enforce the solvency rule `total_allocated + entitlement ≤ total_staked + total_backed` at admission, verify the oracle's attestation of a loss event, keep a first-come line for claims that do not fit today, and stream approved payouts under cooldown, vesting and a daily payout cap. Idle cash is deposited in a DeFindex vault that lends on Blend. Users on Ethereum and Solana stake without a Stellar wallet: they burn USDC through Circle CCTP, and the pool's adapter forwards the mint to a per-user Stellar account that only their own Ethereum or Solana key controls.

This replaces the threat model submitted on 2026-09-01 for `mrkanchwala/safu-soroban` (a single, non-upgradeable XLM pool contract of 2,443 lines). That pool is wound down and is not the audit target.

Out of scope for this model: SAFU's fraud-detection scanner is a separate proprietary system and its code is not in this repository. This model treats the scanner's *output*, a signed verdict, as an input crossing a trust boundary, and models what happens when that output is wrong, delayed, or forged. Also out of scope: Stellar network consensus, the Soroban host and `soroban-sdk` 27.0.0 internals, Circle's CCTP contracts and attestation service, DeFindex and Blend internals, AWS KMS internals, the website, and the separate Ethereum and Solana deployments.

Entities

External entities (outside our control)

	ENTITY	NOTES
C8	**DeFindex vault** (Blend strategy)	Third-party Soroban contract holding deployed idle capital. Untrusted.
C9	**USDC Stellar Asset Contract (SAC)**	Circle's USDC on Stellar. Fixed at construction. Trusted.

	ENTITY	NOTES
C10	**Circle CCTP** (MessageTransmitter, TokenMessenger, attestation service)	Mints USDC on Stellar for a burn on Ethereum or Solana. Checks its own attestation and nonce. Trusted for the mint, nothing else.
C11	**Staker / backer / beneficiary**	End user, on Stellar directly or on Ethereum / Solana through C4. Untrusted input source.
C12	**Stellar network / RPC**	Consensus, ordering, host auth. Trusted platform.

Processes (under our control)

	PROCESS	NOTES
C1	`protection-pool` **contract**	The main system under review. **Upgradable only through governance**: two of three roles plus 7 days in public, never the single-role recovery path.
C2	`covered-registry` **contract**	Binds each covered wallet (stored as a hash) to its staker for good, at most 3 per staker. Written only by the registry writer key.
C3	`cctp-adapter` **contract** (with the `cctp-common` parsing library)	Receives CCTP mints, checks the message, deploys or reuses the sender's C4 account and forwards the USDC. Permissionless to call.
C4	`safu-account` **contract** (one per Ethereum or Solana owner)	Custom Stellar account. `__check_auth` accepts only its owner's Ethereum (secp256k1, EIP-191) or Solana (Ed25519) signature, and only for itself or two pool functions.
C5	**Oracle backend and registry writer**	Off-chain scanner and signing service on a hardened server. The oracle's Ed25519 attestation key and the registry writer key are both held in **AWS KMS** and never leave it.

	PROCESS	NOTES
C6	**Governance roles: admin, co-signer, guardian**	Two of three for every power-granting change and upgrade; admin proposes and co-signer approves settings; admin alone can pause and suspend within limits.
C7	**Frontend**	`safustaking.com`. Client-side; treated as untrusted.

Data storage

STORE	TIER	CONTENTS
C1 `instance()`	pool-wide	Roles, oracle address and public key, asset token, pool cap, totals (staked, backed, pending, allocated, stakers), yield indexes and set-asides, protocol yield, rounding reserve, settings and pending settings, pending governance changes, pause start / end / paused total, line head and tail, payout snapshot, storage version
C1 `persistent()`	per-entity	`StakeRecord` by address; `Claim` by claim id; `BackerRecord` by address; override requests; line and watch-list slots; points balances
C1 `temporary()`	self-expiring	Oracle approval revocations only
C2 `persistent()`	per-wallet	`Registration` by wallet hash; wallet list by staker
C3, C4 `instance()`	per-contract	Adapter: USDC, transmitter, messenger, pool, account WASM hash, mode. Account: owner key, home domain, pool, mode, Solana payout account
AWS KMS	off-chain	Ed25519 oracle attestation key; registry writer key

TTL is **never** used as a security boundary. The 90-day time gate, the claim windows and the 365-day penalty lock are enforced by comparing explicit stored deadlines in contract logic; entry expiry plays no part in them.

State-changing writes bump TTL.

Assets under protection

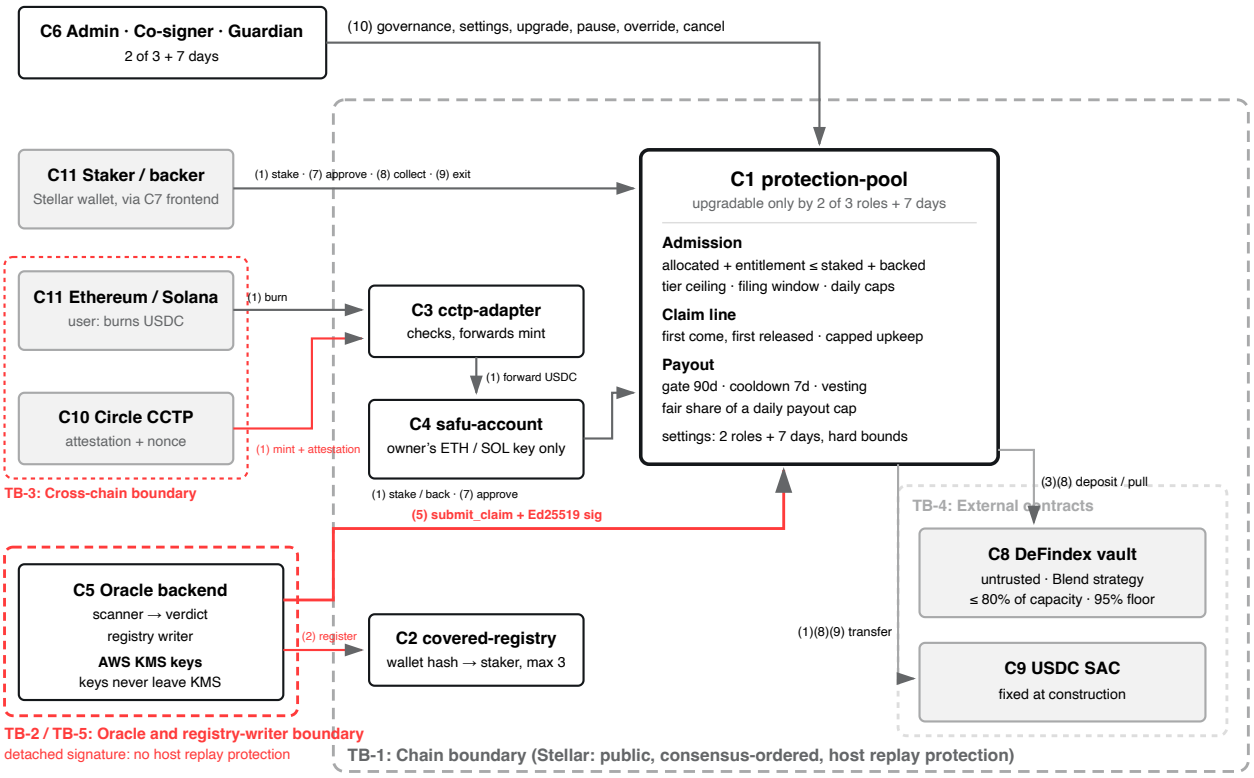
ASSET	WHY IT MATTERS
Staker and backer principal (USDC), liquid or vault-deployed	direct financial loss
Solvency at admission `total_allocated + entitlement ≤ capacity`	the protocol's core promise; a breach means not all payouts can be honoured
Entitlement correctness	an inflated entitlement is a socialised loss borne by everyone else in the pool
Exit availability (`withdraw`, `emergency_exit`, backer drip)	funds locked with no exit is materially a loss even without theft
Claim line order	a line that can be jumped turns timing into a payout advantage
Covered-wallet bindings	a wallet covered twice can be claimed twice
Oracle Ed25519 attestation key	authorises claim admission
Governance role keys	upgrades, settings, vault, treasury, pause, suspension, override

Happy-path flow

1. **Stake.** A Stellar user calls `stake(staker, amount, beneficiary)`. An Ethereum or Solana user burns USDC through CCTP; anyone then calls `cctp-adapter.mint_and_stake(message, attestation)`, which forwards the mint to the user's C4 account, which stakes or backs it. The pool stores `sha256(beneficiary)`, never the plaintext address. A deposit above the maximum stake becomes backing.
2. **Cover.** The owner proves control of each wallet to cover; the registry writer records it in C2.
3. **Idle capital.** Every deposit puts idle cash above a 20% buffer into the vault (C8), up to the deploy ceiling (80% at most).
4. **Loss event.** A covered wallet is drained. SAFU's scanner checks coverage and scores the transaction, producing a verdict: entitlement, tier, hack timestamp.
5. **Attest and submit.** The oracle backend signs the approval payload with the KMS-held Ed25519 key and calls `submit_claim` from the oracle's own Stellar account.

6. **Admit or queue.** The contract verifies caller, host auth and the signature; then checks the tier ceiling, the filing window, solvency against capacity, the daily admission cap and the oracle's daily limit. It admits the claim, or puts it in the line if the blocker is capacity rather than a logic error. Queued claims are released strictly in order.
7. **Staker approves.** After the 90-day time gate the staker calls `approve_claim` (C4 accounts can authorise this one call). The stake is forfeited, points are burned, and a 7-day cooldown and then linear vesting begin.
8. **Collect.** The beneficiary collects with `claim_stream`, within the claim's fair share of each day's payout cap. The contract pulls cash back from the vault first if needed.
9. **Or exit.** The staker calls `withdraw` (to the committed beneficiary) or `emergency_exit` (works while paused). Backers withdraw through a daily drip. Yield can be collected at any time.
10. **Upkeep and exceptions.** Every user call ends with capped upkeep (release the head of the line, move due claims, clear finished entries). Exceptions: admin `cancel_claim`; admin + co-signer `approve_override`; governance changes and settings through their timelocks; permissionless expiries and line release.

Data flow diagram



Numbers in parentheses refer to the happy-path flow steps above.
Solid dark outline = process under our control. Grey = external (entity or contract).
Red = a signed input crossing a trust boundary (oracle verdict, registry write, CCTP attestation).

Trust boundaries

- **TB-1: chain boundary.** Everything crossing it is public, consensus-ordered, and replay-protected by the Soroban host *for authorization entries*.
- **TB-2: oracle attestation boundary.** The one place an off-chain judgement is admitted on-chain. A **detached signed payload** crosses here and does **not** inherit host replay protection, which is why the contract carries its own domain, deadline bound and revocation list *in addition to* `require_auth()`.
- **TB-3: cross-chain boundary.** A CCTP message and Circle's attestation cross here. Circle's transmitter checks the attestation and refuses a used nonce; the adapter checks destination, recipient, source domain and sender, and measures the mint by balance change.
- **TB-4: external contract boundary.** `C1 → C8` and `C1 → C9`. Return values from C8 are never trusted; every redemption is bounded by a minimum output, and shares are measured by balance change.
- **TB-5: registry-writer boundary.** One KMS-held key writes covered-wallet bindings after an off-chain ownership proof. The scanner, not the pool, checks coverage before the oracle signs.

What can go wrong?

Threat table

THREAT	ISSUES
Spoofing	**Spoof.1**: (Step 5) An attacker submits a claim while impersonating the oracle, admitting a fabricated loss event. **Spoof.2**: (Steps 7–9) An attacker impersonates a staker to approve a claim, collect a payout stream, or withdraw principal to an address of their choosing. **Spoof.3**: (Steps 1, 7) An attacker forges an Ethereum or Solana owner's signature to act through that owner's `safu-account`, or uses the account's authority for a call it was never meant to make. **Spoof.4**: (Step 1) An attacker forges or redirects a CCTP mint, crediting a fake deposit or someone else's account. **Spoof.5**: (Step 10) An attacker impersonates a governance role, or a role key is installed at a mistyped or unusable address. **Spoof.6**: (Step 1) An attacker observes a deploy transaction and calls a setup function first. **Spoof.7**: (Steps 5–6) An attacker supplies a chosen claim id to collide with or re-target another wallet's claim.
Tampering	**Tamper.1**: (Step 5) The signed verdict is altered in transit so that entitlement, tier, or the target wallet differ from what the scanner produced. **Tamper.2**: (Step 5) Payload-encoding ambiguity lets two *different* verdicts serialise to identical signed bytes. **Tamper.3**: (Step 5) A signature is replayed on another network, against another pool, or twice on the same pool. **Tamper.4**: (Step 10) A settings change re-prices stakes and claims already in the pool. **Tamper.5**: (Steps 3, 8) The vault reports a false share

THREAT	ISSUES
	price or asset amount, so the pool over-pays on deposit or under-receives on redemption. **Tamper.6**: (Step 3) An attacker inflates the vault's share price before the pool's first deposit. **Tamper.7**: (Steps 8–9) Arithmetic is manipulated through ordering or truncation in tier ceilings, yield indexes or daily shares. **Tamper.8**: (Step 2) A covered wallet is rebound to another staker, or covered again after it was claimed on. **Tamper.9**: (Step 6) The claim line is reordered: a later claim is released before an earlier one.
Repudiation	**Repudiate.1**: (Steps 5–9) A payout, forfeiture, or claim-state transition cannot be attributed after the fact. **Repudiate.2**: (Step 10) A role changes a setting, the oracle, the vault, the treasury, the registry writer, or the code, or pauses the pool, and no monitor can detect it. **Repudiate.3**: (Steps 4–5) The oracle denies having attested to a verdict.
Information Disclosure	**Info.1**: (Step 1) The beneficiary address is exposed on-chain at stake time. **Info.2**: (Step 2) Covered wallets are readable on-chain, telling an attacker which wallets are protected and which are not. **Info.3**: (Step 4) Scanner thresholds or scoring logic are inferable from on-chain data. **Info.4**: (Step 6) Stake amounts, claim history and payouts are publicly readable.
Denial of Service	**DoS.1**: (Steps 5, 9) A compromised oracle files claims against many stakers at once; a staker with an open claim cannot exit. **DoS.2**: (Step 6) Claim flooding, or a mass incident, crowds genuine claims out. **DoS.3**: (Step 8) A coordinated payout run drains the pool faster than it can absorb.

THREAT	ISSUES
	DoS.4: (Steps 5–8) A pause, or days the vault cannot pay, run out a claim's filing, approval or collection window. **DoS.5**: (Steps 8–9) A payment cannot be funded because cash is in the vault. **DoS.6**: (all) Storage archival strands a record, or an attacker-sized loop makes a call too expensive. **DoS.7**: (Step 1) A CCTP deposit is lost when the pool refuses it. **DoS.8**: (Step 10) An endless pause holds every claim.
Elevation of Privilege	**Elevation.1**: (all) The contract is upgraded to a malicious implementation that drains principal. **Elevation.2**: (Step 10) One role takes over the others. **Elevation.3**: (Step 10) The admin turns configuration into theft of principal. **Elevation.4**: (Step 10) The 2-of-2 override pays a colluding claimant. **Elevation.5**: (Steps 7–9) A false claimant whose claim is cancelled keeps what was paid and leaves at once with the full stake. **Elevation.6**: (Step 3) The pool's authority leaks into a vault sub-invocation. **Elevation.7**: (Step 1) The beneficiary is set to a privileged party, the pool, or its vault.

What are we going to do about it?

Every mitigation below is **implemented in the tree being submitted for audit** unless explicitly labelled **ACCEPTED RISK** or **OPERATIONAL**.

THREAT	ISSUES AND REMEDIATIONS
Spoofing	**Spoof.1**: impersonating the oracle. **Spoof.1.R.1**: `submit_claim` accepts only the stored oracle address, checked first, then `caller.require_auth()`. **Spoof.1.R.2**: `ed25519_verify` checks a signature over the full verdict against the stored oracle public key. The caller check binds *who submitted*, the signature binds *who attested*: an attacker needs both the Stellar account and the KMS key. **Spoof.1.R.3**: the admin's former unsigned filing path is removed. One admin key alone cannot file a claim; manual corrections go through the 2-of-2 override. **Spoof.2**: impersonating a staker. **Spoof.2.R.1**: `approve_claim` and `claim_stream` require the claim wallet's signature. **The victim must authorise their own payout.** **Spoof.2.R.2**: `claim_stream`, `withdraw` and `claim_yield` recompute `sha256(beneficiary)` and compare it with the hash committed at stake time. Funds reach only the address nominated before the incident. **Spoof.3**: forging a cross-chain owner. **Spoof.3.R.1**: `__check_auth` verifies an Ethereum signature by secp256k1 recovery over the EIP-191 digest of the payload (recovery id checked), or a Solana Ed25519 signature, against the owner key fixed when the account was deployed. **Spoof.3.R.2**: every authorization context is checked: the account may authorise only itself and the pool's `approve_claim` and `revoke_approval`. Any other contract or function is refused (`ContextNotAllowed`). **Spoof.3.R.3**: the account address is derived from the owner key and home chain, so each owner has exactly one account. **Spoof.4**: forged or redirected CCTP mint. **Spoof.4.R.1**: Circle's transmitter verifies the attestation and refuses a used nonce. **Spoof.4.R.2**: before receiving, the adapter refuses a

THREAT	ISSUES AND REMEDIATIONS
	message whose destination caller or mint recipient is not the adapter, or whose source domain is not Ethereum or Solana; a refused call leaves the message unconsumed. **Spoof.4.R.3**: the receiving account is derived from the message's sender, and the amount is the measured USDC balance change, never a field in the message. **Spoof.5**: impersonating a governance role. **Spoof.5.R.1**: every propose, approve, cancel and settings step requires the acting role's signature; the role a change replaces never counts toward it. **Spoof.5.R.2**: a new admin, co-signer, guardian, oracle or registry writer key must sign its own installation, so a mistyped address cannot be installed. **Spoof.6**: deploy front-running. **Spoof.6.R.1**: all five contracts set up in `__constructor`, inside the deploy transaction itself. There is no window between deploy and setup. **Spoof.7**: claim-id forgery. **Spoof.7.R.1**: `claim_id = sha256(wallet ‖ tx_hash)` is computed on-chain in every path, including overrides.
Tampering	**Tamper.1**: verdict altered. **Tamper.1.R.1**: every decision field is inside the signed payload: domain string, pool address, network id, wallet, tx hash, entitlement, tier, hack timestamp, deadline. **Tamper.2**: encoding ambiguity. **Tamper.2.R.1**: each field is XDR-serialised on its own into a self-delimiting `ScVal` before concatenation, rather than packed. Distinct verdicts cannot produce identical bytes. **Tamper.3**: replay. **Tamper.3.R.1**: the pool address and network id are read live from the host and signed, so a signature is valid for one pool on one network only. **Tamper.3.R.2**: the claim id is the nonce (`ClaimAlreadyExists`); the deadline is at most 24 hours; the admin can revoke a signed approval before it is used, and the revocation outlives every approval it could revoke (compile-time assertion). **Tamper.4**: settings re-pricing people already in. **Tamper.4.R.1**: values a stake or claim depends on (tier ceilings, time gate, filing window, vesting) are copied onto it when it is created. A tier ceiling cut applies to new

THREAT	ISSUES AND REMEDIATIONS
	stakes only. **Tamper.4.R.2**: every setting has hard bounds and order rules (min ≤ max, bands in order, tier A ≥ B ≥ C, pause ≤ gap, filing window ≤ gap), checked at proposal and again at execution, after a 7-day public timelock. **Tamper.5**: vault reports false values. **Tamper.5.R.1**: shares and assets are measured by balance change, never by a rate the vault returns. **Tamper.5.R.2**: every pull carries a minimum of 95% of book value, enforced by the vault's own `min_amounts_out`; a larger loss is refused and only the admin can redeem at a lower floor. **Tamper.6**: first-deposit share inflation. **Tamper.6.R.1**: the vault setup script makes the vault's first deposit from a separate seeder wallet, so the shares DeFindex locks are never the pool's, and the pool's automatic deposits derive their floor from its own earlier position. **Tamper.7**: arithmetic manipulation. **Tamper.7.R.1**: multiply before divide in tier ceilings, yield indexes and daily shares; rounding favours the pool. **Tamper.7.R.2**: the release build aborts on overflow; allocation releases use saturating subtraction. **Tamper.7.R.3**: a rounding reserve that anyone can fund and nothing can take out covers a last-withdrawer gap of at most 10,000 units (0.001 USDC). **Tamper.8**: covered wallet rebound or covered twice. **Tamper.8.R.1**: the registry binds a wallet hash to its staker for good, at most 3 per staker, written only by the writer key. **Tamper.8.R.2**: an address with an approved claim can never stake again, so a drained wallet cannot be covered by the same stake twice. **Tamper.9**: line reordering. **Tamper.9.R.1**: queued claims are released strictly in the order they joined, and a new claim joins the back while anyone waits, even if it would fit alone. Release is permissionless and also runs in upkeep.
Repudiation	**Repudiate.1**: payouts unattributable. **Repudiate.1.R.1**: 58 typed `#[contractevent]` types across the five contracts cover every money movement and claim transition (submitted, queued, released,

THREAT	ISSUES AND REMEDIATIONS
	approved, streamed, cancelled, expired, overrides), stakes, withdrawals, exits, backer deposits and drips, yield, vault moves and CCTP forwards. **Repudiate.1.R.2**: stake, claim and backer state is readable at any time through public views. **Repudiate.2**: administrative action undetectable. **Repudiate.2.R.1**: every governance step (proposed, approved, executed, cancelled), every settings step, pause, unpause, suspension and treasury payout emits an event a monitor can subscribe to. (The September model's open item, admin setters with no events, is closed.) **Repudiate.3**: oracle denies attestation. **Repudiate.3.R.1**: the Ed25519 signature over the full payload is the evidence, verifiable by anyone against the published oracle key. **Repudiate.3.R.2**: signing happens inside AWS KMS, which logs every signing call outside the reach of the application host.
Information Disclosure	**Info.1**: beneficiary exposed. **Info.1.R.1**: only `sha256(beneficiary)` is stored; the address appears on-chain only when paid. `set_beneficiary` lets the staker rotate it before forfeiture. **Info.2**: covered wallets readable. **Info.2.R.1**: the registry stores a hash of each covered wallet, not the address. **Info.3**: scanner logic inferable. **Info.3.R.1**: the contract receives only a finished verdict; no scoring logic is on-chain or in this repository. **Info.4**: pool state is public. **ACCEPTED RISK.** **Info.4.R.1**: inherent to a transparent ledger and to the pool's own auditability promise.
Denial of Service	**DoS.1**: oracle griefing. **ACCEPTED RISK: bounded, remedy exists and is penalty-free; residual is operational.** **DoS.1.R.1**: bounded by the oracle's daily limit, one tenth of stakers per day. **DoS.1.R.2**: nothing is forfeited until the staker approves their own claim. **DoS.1.R.3**: admin `cancel_claim` clears a claim the

THREAT	ISSUES AND REMEDIATIONS
	staker never approved with no penalty. **DoS.1.R.4**: **OPERATIONAL:** clearing depends on the admin acting. **DoS.2**: flooding or a mass incident. **DoS.2.R.1**: a claim that does not fit today joins the line instead of failing; no claim is cleared from the line while its stake is live. **DoS.2.R.2**: one live and one queued claim per stake; every claim needs an oracle signature and a stake past the 90-day gate. **DoS.3**: payout run. **DoS.3.R.1**: admission checks `total_allocated + entitlement ≤ capacity` and a daily admission cap tiered by utilisation. **DoS.3.R.2**: payouts are spread by cooldown, vesting and a daily payout cap; each claim gets a fair share by what it is still owed, computed against its activation snapshot, so shrinking the pool cannot slow it. **DoS.4**: windows running out. **DoS.4.R.1**: paused time and days the vault could not pay are added back to the approval and collection windows; every expiry refuses while paused. **DoS.4.R.2**: no claim can be filed while paused, so paused time since the hack is added back to the filing window. The filing window can never be set longer than the gap between pauses, so at most one pause falls inside it. **DoS.5**: payment unfunded. **DoS.5.R.1**: every outbound payment first pulls the shortfall plus a refill of the 20% cash buffer from the vault, in the same call; this works while paused. **DoS.5.R.2**: if the vault cannot pay, the payment returns a typed error and the claim's clock does not run that day. **DoS.6**: archival and resource exhaustion. **DoS.6.R.1**: records are bumped on write; TTL is never a deadline. **DoS.6.R.2**: every loop has a fixed cap: 16 empty line slots, 8 watch-list slots, 2 upkeep entries, 8 pending backer deposits, 3 wallets, 4 roles. Upkeep checks before it writes and never fails the caller. **DoS.7**: CCTP deposit refused. **DoS.7.R.1**: if the pool refuses, the account keeps the USDC (`Held` event) and its owner can stake or back it

THREAT	ISSUES AND REMEDIATIONS
	later. An adapter call refused before receipt leaves the message unconsumed. **DoS.8**: endless pause. **DoS.8.R.1**: a pause lasts at most 30 days and ends on its own; the next pause needs a gap of more than 30 days after the last one ended. **DoS.8.R.2**: `emergency_exit` and backer withdrawals work while paused.
Elevation of Privilege	**Elevation.1**: malicious upgrade. **ACCEPTED RISK, stated plainly: two of three roles control the pool after 7 days.** **Elevation.1.R.1**: an upgrade is a governance change: one role proposes, a second approves, 7 days in public, then anyone executes. It can never use the single-role recovery path. **Elevation.1.R.2**: stakers can leave during those 7 days, also while paused. Storage is versioned (`migrate`), so an upgrade does not silently misread stored data. **Elevation.2**: one role takes over. **Elevation.2.R.1**: roles must be distinct. One remaining role can replace the others only after 90 days, and any other working role, including the one being replaced, can cancel it. **Elevation.3**: admin turns configuration into theft. **Elevation.3.R.1**: no function sends staker or backer principal anywhere but its owner's committed address. **Elevation.3.R.2**: the treasury takes only the protocol's own yield share, only above everything the pool owes, and never while paused. **Elevation.3.R.3**: a pause or suspension delays, never takes: a suspended staker can still withdraw, and suspensions by the admin alone lapse after 30 days. **Elevation.4**: collusive override. **ACCEPTED RISK, bounded.** **Elevation.4.R.1**: the override needs admin and co-signer, still needs the staker's own approval, and stays inside the tier ceiling, capacity, daily caps, cooldown and vesting. Every step is on-chain. **Elevation.5**: cancelled false claim keeps its gains. **Elevation.5.R.1**: cancelling an active claim restores only the stake minus what was already paid; a claim paid up to the stake leaves it forfeited.

THREAT	ISSUES AND REMEDIATIONS
	Elevation.5.R.2: the rest is locked for 365 days, on both `withdraw` and `emergency_exit`. **Elevation.6**: authority leaking into vault calls. **Elevation.6.R.1**: each vault call declares an exact authorization tree for that call only. **Elevation.7**: privileged or circular beneficiary. **Elevation.7.R.1**: the beneficiary cannot be the oracle, admin, co-signer, the pool or its vault, at stake time or when changed.

Summary of open items

ID	SEVERITY	ITEM	DISPOSITION
Elevation.1	n/a	Two of three roles control the pool after 7 days, including upgrades	**Accepted risk**, disclosed (`docs/AUDITOR_DISCLOSURES.md` §2, §3)
Elevation.4	n/a	Collusive override: bounded, observable	**Accepted risk**
DoS.1	LOW-MEDIUM	Oracle griefing (one tenth of stakers per day); cleared by admin `cancel_claim`, penalty-free	**Accepted + disclosed**: operational dependency
DoS.4.R.2	LOW	A stake keeps the filing window it was opened with. If the window and then the pause gap are both lowered later, an older stake with a longer window could see two pauses inside it and gets back only the later one	**Disclosed** (§2); needs two timelocked setting changes
TB-5	n/a	Coverage is checked off-chain before the oracle signs; the pool trusts the signed verdict	**Accepted, by design**: one trust root (the oracle)

ID	SEVERITY	ITEM	DISPOSITION
Info.4	n/a	Stake and claim data are public	**Accepted risk**, inherent
Liquidity	n/a	After large claims the last withdrawer can wait for new money	**Accepted, by design** (§1)

Additional disclosures

Pool liquidity model. Stakers are always owed their full principal; a claim larger than the claimant's own stake is paid from pooled liquidity and covered over time by new stake, yield and backers. Measured with contract tests and simulations in `docs/AUDITOR_DISCLOSURES.md` §1.

Error naming. `PoolError::CallerNotOracleOrAdmin` still says "admin" although only the oracle may now file a claim. The name is kept so existing clients keep decoding the same error code.

Assurance evidence

CONTROL	RESULT	SCOPE NOTE
Unit and integration tests	**553 passing** (pool 486, adapter 30, account 16, registry 13, common 8)	`main`, 2026-10-20
Fuzzing	**6 targets, 76,812 runs, no failures** (solvency, backers, override, registry, account signatures, CCTP flows)	Build server, capped container
Mutation testing	**Every function changed by the fixes re-checked:** 177 caught, 11 missed; 7 closed with new tests (each confirmed to fail on its mutant), 4 change nothing	GitHub Actions, `.github/workflows/mutants.yml`; every remaining miss explained in `docs/MUTATION_EQUIVALENTS.md`
Scout (CoinFabrik)	**Run on the fixed code; every medium result reviewed**, none open. Critical results are all the overflow lint, covered by `overflow-checks = true`	`docs/SCOUT_TRIAGE.md`
Coverage	**96.76% line**	Measured 2026-10-19
`cargo audit`	**0 known vulnerabilities across 219 dependencies**	Measured 2026-10-19

CONTROL	RESULT	SCOPE NOTE
`cargo clippy`	0 warnings in contract code	2026-10-20
Internal contract review	0 critical / high open; 2 medium and 3 low found on 2026-10-20 and fixed with tests	Internal

Did we do a good job?

Has the data flow diagram been referenced since it was created?

Yes: adding the cross-chain path to the diagram is what made TB-3 and TB-5 separate boundaries rather than footnotes. A CCTP mint and a registry write are both signed inputs from outside Stellar, like the oracle verdict, but each is checked by a different party (Circle, and our own writer key), and drawing them in red next to TB-2 is what prompted the question of what the pool itself checks at each one.

Did the STRIDE model uncover any new design issues or concerns that had not been previously addressed?

Yes: walking the Denial of Service and Elevation of Privilege columns against the pause and cancel paths surfaced two medium issues:

- a pause right after a hack could run out a real victim's 30-day filing window, because claims cannot be filed while paused and paused time was not added back (now DoS.4.R.2);
- cancelling a partly paid false claim restored the full stake, and `emergency_exit` skipped the 365-day lock (now Elevation.5).

Three low items came out of the same pass: unpaid staker yield could be wiped after a vault loss, the treasury could take its share while paused, and the pool could be named as a payout address. All five are fixed with tests in the tree being submitted. A governance gap found while reviewing Scout output (a new registry writer installed without its own signature) was fixed the same day (Spoof.5.R.2).

Did the treatments adequately address the issues identified?

Mostly, and the exceptions are named. Every Spoofing, Tampering, Repudiation and Information Disclosure issue is treated in code. The September model's one open code item, admin actions with no events, is closed. What remains is accepted rather than treated: governance by two of three roles including upgrades (Elevation.1), the bounded collusive override (Elevation.4), the operational dependency in DoS.1, and the trust placed in the oracle's coverage check (TB-5). The filing-window edge in DoS.4.R.2 is disclosed rather than closed, because closing it fully would need a pause history that does not fit in the contract's size limit.

Have additional issues been found after the threat model?

None so far: the model is current as of `main`, contract code at `6e61cf4`. It will be re-run against the post-audit tree before the mainnet migration.

Any additional thoughts on the threat modelling process?

Checking each fix against every path that touches the same state (pause, cancel, exit, yield) found more than reviewing each function alone did: both medium issues sat where two of those paths meet. We keep one list of these cross-path rules and check every new change against it.

Prepared by SAFU as a self-assessment for the security audit and the SCF Soroban Audit Bank intake. Open findings are disclosed here rather than remediated silently; accepted risks are also published in the repository's `docs/AUDITOR_DISCLOSURES.md`*.*